



Singularry ListaFacet

AUDIT REPORT

Version 1.0.0

Serial No. 2026040600012023

Presented by Fairyproof

April 6, 2026

01. Introduction

This document includes the results of the audit performed by the Fairyproof team on the Singularry ListaFacet project.

Audit Start Time:

March 17, 2026

Audit End Time:

April 6, 2026

Audited Code's Github Repository:

<https://github.com/singularry/slywallet-contracts-foundry/tree/main>

Audited Code's Github Commit Number When Audit Started:

5a4ebe433f85bf7b7d828ba58de5b454437f982e

Audited Code's Github Commit Number When Audit Ended:

329408e48005506efdc9a30d6b9219cc75f3210c

Audited Source Files:

The source files audited include all the files as follows:

```
src/slywallet/facets/lista
├── ISLYWalletListaManagementFacet.sol
├── ListaFacetBase.sol
├── ListaTypes.sol
├── SLYWalletListaBnbLoopingFacet.sol
├── SLYWalletListaDirectFacet.sol
├── SLYWalletListaManagementFacet.sol
├── SLYWalletListaMonitoringFacet.sol
├── SLYWalletListaStableLoopingFacet.sol
├── SLYWalletListaStorage.sol
├── libraries
│   ├── LibListaExecution.sol
│   ├── LibListaFees.sol
│   └── LibListaOracle.sol
2 directories, 12 files
```

The goal of this audit is to review Singularry's solidity implementation for its ListaFacet function, study potential security vulnerabilities, its general design and architecture, and uncover bugs that could compromise the software in production.

We make observations on specific areas of the code that present concrete problems, as well as general observations that traverse the entire codebase horizontally, which could improve its quality as a whole.

This audit only applies to the specified code, software or any materials supplied by the Singularry team for specified versions. Whenever the code, software, materials, settings, environment etc is changed, the comments of this audit will no longer apply.

— Disclaimer

Note that as of the date of publishing, the contents of this report reflect the current understanding of known security patterns and state of the art regarding system security. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. If the audited source files are smart contract files, risks or issues introduced by using data feeds from offchain sources are not extended by this review either.

Given the size of the project, the findings detailed here are not to be considered exhaustive, and further testing and audit is recommended after the issues covered are fixed.

To the fullest extent permitted by law, we disclaim all warranties, expressed or implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We do not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

— Methodology

The above files' code was studied in detail in order to acquire a clear impression of how the its specifications were implemented. The codebase was then subject to deep analysis and scrutiny, resulting in a series of observations. The problems and their potential solutions are discussed in this document and, whenever possible, we identify common sources for such problems and comment on them as well.

The Fairyproof auditing process follows a routine series of steps:

1. Code Review, Including:
 - Project Diagnosis

Understanding the size, scope and functionality of your project's source code based on the specifications, sources, and instructions provided to Fairyproof.

- Manual Code Review

Reading your source code line-by-line to identify potential vulnerabilities.

- Specification Comparison

Determining whether your project's code successfully and efficiently accomplishes or executes its functions according to the specifications, sources, and instructions provided to Fairyproof.

2. Testing and Automated Analysis, Including:
 - Test Coverage Analysis

Determining whether the test cases cover your code and how much of your code is exercised or executed when test cases are run.

- Symbolic Execution

Analyzing a program to determine the specific input that causes different parts of a program to execute its functions.

3. Best Practices Review

Reviewing the source code to improve maintainability, security, and control based on the latest established industry and academic practices, recommendations, and research.

— Structure of the document

This report contains a list of issues and comments on all the above source files. Each issue is assigned a severity level based on the potential impact of the issue and recommendations to fix it, if applicable. For ease of navigation, an index by topic and another by severity are both provided at the beginning of the report.

— Documentation

For this audit, we used the following source(s) of truth about how the token issuance function should work:

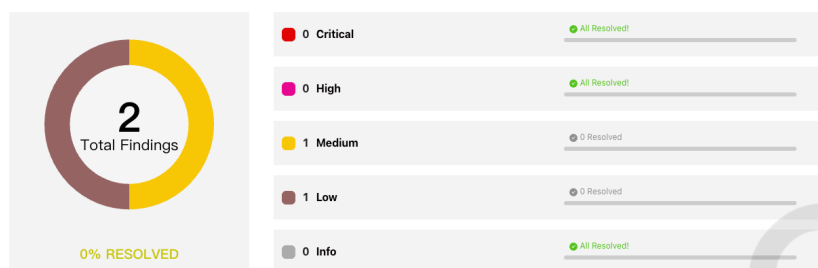
Website: <https://singularray.org/>

Source Code: <https://github.com/singularray/slywallet-contracts-foundry/tree/main>

These were considered the specification, and when discrepancies arose with the actual code behavior, we consulted with the Singularray team or reported an issue.

— Comments from Auditor

Serial Number	Auditor	Audit Time	Result
2026040600012023	Fairyproof Security Team	Mar 17, 2026 - Apr 6, 2026	Medium Risk



Summary:

The Fairyproof security team used its auto analysis tools and manual work to audit the project. During the audit, one issue of medium-severity and one issue of low-severity were uncovered. The Singularray team acknowledged all the issues.

02. About Fairyproof

[Fairyproof](#) is a leading technology firm in the blockchain industry, providing consulting and security audits for organizations. Fairyproof has developed industry security standards for designing and deploying blockchain applications.

03. Introduction to Singularray

Singularray is a platform of creating next-generation financial systems with intelligent, autonomous protocols and seamless cross-chain operations.

The above description is quoted from relevant documents of Singularray.

04. Major functions of audited code

The main function of audit code is:

The Singularray wallet offers a variety of yield strategies, all based on listaDAO's slisBNB token. These strategies include multi-cycle strategies and single-staking multi-DeFi strategies, which involve token swaps. Users should be aware that when participating in different DeFi protocols, the liquidity of the tokens swapped and the staking/lending rates of the DeFi protocols will affect the final yield.

05. Coverage of issues

The issues that the Fairyproof team covered when conducting the audit include but are not limited to the following ones:

- Access Control
- Admin Rights
- Arithmetic Precision
- Code Improvement
- Contract Upgrade/Migration
- Delete Trap
- Design Vulnerability
- DoS Attack
- EOA Call Trap
- Fake Deposit
- Function Visibility
- Gas Consumption
- Implementation Vulnerability
- Inappropriate Callback Function
- Injection Attack
- Integer Overflow/Underflow
- IsContract Trap
- Miner's Advantage
- Misc
- Price Manipulation
- Proxy selector clashing
- Pseudo Random Number
- Re-entrancy Attack
- Replay Attack
- Rollback Attack
- Shadow Variable
- Slot Conflict
- Token Issuance
- Tx.origin Authentication
- Uninitialized Storage Pointer

06. Severity level reference

Every issue in this report was assigned a severity level from the following:

Critical severity issues need to be fixed as soon as possible.

High severity issues will probably bring problems and should be fixed.

Medium severity issues could potentially bring problems and should eventually be fixed.

Low severity issues are minor details and warnings that can remain unfixed but would be better fixed at some point in the future.

Informational is not an issue or risk but a suggestion for code improvement.

07. Major areas that need attention

Based on the provided source code the Fairyproof team focused on the possible issues and risks related to the following functions or areas.

- Function Implementation

We checked whether or not the functions were correctly implemented.
We didn't find issues or risks in these functions or areas at the time of writing.

- Access Control

We checked each of the functions that could modify a state, especially those functions that could only be accessed by owner or administrator
We didn't find issues or risks in these functions or areas at the time of writing.

- Token Issuance & Transfer

We examined token issuance and transfers for situations that could harm the interests of holders.
We didn't find issues or risks in these functions or areas at the time of writing.

- State Update

We checked some key state variables which should only be set at initialization.
We didn't find issues or risks in these functions or areas at the time of writing.

- Asset Security

We checked whether or not all the functions that transfer assets were safely handled.
We didn't find issues or risks in these functions or areas at the time of writing.

- Miscellaneous

We checked the code for optimization and robustness.
We found some issues, for more details please refer to [FP-1,FP-2] in "09. Issue description".

08. List of issues by severity

Index	Title	Issue/Risk	Severity	Status
FP-1	Some strategies are significantly affected by external DEX liquidity	Design Vulnerability	Medium	Acknowledged
FP-2	User can only activate one Lista strategy at a time.	Design Vulnerability	Low	Acknowledged

09. Issue descriptions

[FP-1] Some strategies are significantly affected by external DEX liquidity

Design Vulnerability

Medium

Acknowledged

Issue/Risk: Design Vulnerability

Description:

Some strategies require token swaps on PancakeV3. However, if users have a large amount of capital involved or the corresponding liquidity pool on PancakeV3 is insufficient, user returns may decrease or some strategies may fail to execute successfully.

Recommendation:

Remind users to pay attention to changes in exchange slippage and yield rate of external defi protocol.

Update/Status:

The team has acknowledged the issue.

[FP-2] User can only activate one Lista strategy at a time.

Design Vulnerability

Low

Acknowledged

Issue/Risk: Design Vulnerability

Description:

In the current design, only one strategy can be activated at a time, and a new position for another strategy can only be created after the position created by the previous strategy is closed.

Recommendation:

Inform users of the relevant design intent.

Update/Status:

The team has acknowledged the issue.

10. Recommendations to enhance the overall security

We list some recommendations in this section. They are not mandatory but will enhance the overall security of the system if they are adopted.

- Consider managing the owner's access control with great care and transferring it to a multi-sig wallet or DAO when necessary.

11. Appendices

11.1 Unit Test

1. ListaDirect.t.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
```

```

import {ListaTestBase} from "../ListaTestBase.sol";
import {MarketParams,Id} from "src/common/lista/IListaLending.sol";
import
{ListaMarketConfig,ListaStrategyType,ListaCollateralType,ListaBorrowType,Lista
PositionStatus} from "src/slywallet/facets/lista/ListaTypes.sol";
import {CreateListaPositionParams} from
"src/slywallet/facets/lista/ListaTypes.sol";
import {SLYWalletListaDirectFacet} from
"src/slywallet/facets/lista/SLYWalletListaDirectFacet.sol";
import {SLYWalletListaStorage} from
"src/slywallet/facets/lista/SLYWalletListaStorage.sol";
import {IDiamondCut} from "src/common/interfaces/IDiamondCut.sol";
import {IMoolah,Id,MarketParamsLib} from "src/common/lista/IListaLending.sol";
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {MockSmartLending} from "src/common/mocks/MockSmartLending.sol";
import {MockERC20} from "src/common/mocks/MockERC20.sol";
import {console} from "forge-std/console.sol";

interface IBNBVault {
    function supplyQueueLength() external view returns (uint256);
    function supplyQueue(uint256) external view returns (bytes32);
}

interface IListaVault {
    function poke(address token) external;
    function drip(address token) external;
    function urns(bytes32 id, address user) external view returns(uint256
ink,uint256 art);
    function ilks(bytes32 id) external view returns(uint256 art,uint256
rate,uint256 pot,uint256 line,uint256 dust);
}

contract ListaDirectTest is ListaTestBase {
    using MarketParamsLib for MarketParams;
    SLYWalletListaDirectFacet directFacet;
    address internal constant IRM =
0xFe7dAe87Ebb11a7BEB9F534BB23267992d9cDe7c;
    address internal constant ORACLE_ADAPTOR =
0x21650E416dC6C89486B2E654c86cC2c36c597b58; // slisBNB markets
    address internal constant ORACLE_ADAPTOR_V2 =
0xf3afD82A4071f272F403dC176916141f44E6c750;
    address internal constant ORACLE_ADAPTOR_PENDLE =
0xf9135dCC8c520331c73e8538BC22e409Af8BA265;
    address internal constant PT_TOKEN =
0xe052823b4aefc6e230FAf46231A57d0905E30AE0;
    address internal constant RE7LABS_VAULT =
0xC41f2Ba7102e9F9F2d603eb951F955aE205ed272;
    address internal constant MY_BINANCE_WALLET =
0x59Ece28c2BCa3B39712cece45B691a934A97cc31; // a test binance wallet
    address internal constant LISUSD =
0x0782b6d8c4551B9760e74c0545a9bCD90bdc41E5;
    address internal constant LISTA_VAULT =
0x33A34eAB3ee892D40420507B820347b1cA2201c4;
    address internal constant LISTA_CDP =
0xB68443Ee3e828baD1526b3e0Bdf2Dfc6b1975ec4;
    bytes32 internal constant SLISBNB_ILK =
bytes32(uint256(0x536e424e4200000000000000000000000000000000000000000000000000000000));
    uint256 constant RAY = 10 ** 27;

    function setUp() virtual override public {
        vm.createSelectFork(url, blockNo);
        super.setUp();
        // add direct facet
        directFacet = new SLYWalletListaDirectFacet();
        IDiamondCut.FacetCut[] memory cut = new IDiamondCut.FacetCut[](1);
        cut[0] = IDiamondCut.FacetCut({
            facetAddress: address(directFacet),
            action: IDiamondCut.FacetCutAction.Add,
            functionSelectors: get_Lista_direct_selectors()
        });
        wallet.diamondCut(cut, address(0), new bytes(0));
        directFacet = SLYWalletListaDirectFacet(payable(address(wallet)));
    }
}

```

```

function test_strategy_SLISBNB_LISUSD_USD1() public {
    // 1. config the strategy, no market integrated
    ListaStrategyType strategyType =
ListaStrategyType.SLISBNB_LISUSD_USD1;
    ListaCollateralType collateralType = ListaCollateralType.SLISBNB;
    ListaBorrowType borrowType = ListaBorrowType.LISUSD;
    managementFacet.configureStrategyInfo(strategyType, collateralType,
borrowType);
    // 2. create a position
    uint256 initialAmount = 1 ether;
    vm.deal(address(this), initialAmount * 2);
    CreateListaPositionParams memory pos_params =
CreateListaPositionParams({
        strategyType: ListaStrategyType.SLISBNB_LISUSD_USD1,
        collateralType: ListaCollateralType.SLISBNB,
        borrowType: ListaBorrowType.LISUSD,
        initialAmount: initialAmount,
        targetLoops: 5,
        targetLtvBps: 6000,
        slippageBps: DEFAULT_SLIPPAGE,
        pendleMarket: address(0),
        pendleMinPtOut: 0,
        externalVault: address(0),
        useAsterProtocol: false,
        launchpoolDelegate: MY_BINANCE_WALLET
    });
    uint256 positionId = managementFacet.createListaPosition(value:
initialAmount)(pos_params);
    assertEq(positionId, 0);
    // 2. execute position
    directFacet.executeSlisBnbLisUsdUsd1WithDelegation(positionId,
RE7LABS_VAULT, MY_BINANCE_WALLET);
    // 3. check the status
    SLYWalletListaStorage.ListaPosition memory position =
managementFacet.getListaPosition(positionId);
    assertEq(position.id, 0);
    assertEq(position.targetLoops, 5);
    assertEq(position.currentLoops, 1);
    assertEq(uint8(position.status), uint8(ListaPositionStatus.ACTIVE));
    console.log("totalCollateral:", position.totalCollateral);
    console.log("totalBorrow:", position.totalBorrowed);
    console.log("lastHealthFactor:", position.lastHealthFactor);
    console.log("entrySlisBnbRate:", position.entrySlisBnbRate);
    console.log("entryBorrowRateBps:", position.entryBorrowRateBps);
    console.log("externalVaultShares:", position.externalVaultShares);
    console.log("status:", uint8(position.status));

    // wrap block
    vm.warp(block.timestamp + 150 seconds);
    vm.roll(block.number + 100);
    vm.prank(USD1_WHALE);
    // transfer usd1 to wallet for paying debt
    ERC20(USD1).transfer(address(wallet), 1 * 1e18);
    // 4. unwind
    uint256 collateralReturned =
directFacet.unwindSLISBNB_LISUSD_USD1(positionId);
    console.log("Collateral returned from unwind:", collateralReturned);
    // 5. check balance of slisBNB
    uint256 slisBnbBalance = ERC20(SLIS_BNB).balanceOf(address(wallet));
    console.log("slisBNB balance after unwind:", slisBnbBalance);
    assertEq(slisBnbBalance, collateralReturned);
    // check status of position
    position = managementFacet.getListaPosition(positionId);
    assertEq(position.id, 0);
    assertEq(position.targetLoops, 5);
    assertEq(position.currentLoops, 1);
    assertEq(uint8(position.status), uint8(ListaPositionStatus.CLOSED));
    console.log("totalCollateral:", position.totalCollateral);
    console.log("totalBorrow:", position.totalBorrowed);
    console.log("lastHealthFactor:", position.lastHealthFactor);
    console.log("entrySlisBnbRate:", position.entrySlisBnbRate);
    console.log("entryBorrowRateBps:", position.entryBorrowRateBps);
    console.log("externalVaultShares:", position.externalVaultShares);
    console.log("status:", uint8(position.status));
}

```

```

// create a new position without initialAmount should be successful
positionId = managementFacet.createListaPosition(pos_params);
assertEq(positionId,1,"positionId should be 1");

}

function test_strategy_SLISBNB_USD1_MARKET() public {
    // 1. config market SLISBNB_USD1_MARKET
    MarketParams memory params = MarketParams({
        loanToken: USD1,
        collateralToken: SLIS_BNB,
        oracle: ORACLE_ADAPTOR,
        irm: IRM,
        lltv: 0.7e18 // 70%
    });
    managementFacet.addListaMarketConfig(
        params,7000,MIN_LTV_BPS
    );
    // 2. config strategy
    ListaStrategyType strategyType =
ListaStrategyType.SLISBNB_USD1_MARKET;
    ListaCollateralType collateralType = ListaCollateralType.SLISBNB;
    ListaBorrowType borrowType = ListaBorrowType.USD1;
    managementFacet.configureStrategyInfoWithMarket(
        strategyType,
        collateralType,
        borrowType,
        params
    );
    // 3. create a position
    uint256 initialAmount = 1 ether;
    vm.deal(address(this), initialAmount * 2);
    CreateListaPositionParams memory pos_params =
CreateListaPositionParams({
        strategyType: ListaStrategyType.SLISBNB_USD1_MARKET,
        collateralType: ListaCollateralType.SLISBNB,
        borrowType: ListaBorrowType.USD1,
        initialAmount: initialAmount,
        targetLoops: 5,
        targetLtvBps:6000,
        slippageBps: DEFAULT_SLIPPAGE,
        pendleMarket:address(0),
        pendleMinPtOut:0,
        externalVault:address(0),
        useAsterProtocol:false,
        launchpoolDelegate:address(0)
    });
    uint256 positionId = managementFacet.createListaPosition{value:
initialAmount}(pos_params);
    assertEq(positionId, 0);
    // 4. execute position
    directFacet.executeSlisBnbUsd1Market(positionId, RE7LABS_VAULT);
    // 5. check the status of position
    SLWWalletListaStorage.ListaPosition memory position =
managementFacet.getListPosition(positionId);
    assertEq(position.id, 0);
    assertEq(position.targetLoops, 5);
    assertEq(position.currentLoops, 1);
    assertEq(uint8(position.status), uint8(ListaPositionStatus.ACTIVE));
    console.log("totalCollateral:", position.totalCollateral);
    console.log("totalBorrow:", position.totalBorrowed);
    console.log("lastHealthFactor:", position.lastHealthFactor);
    console.log("entrySlisBnbRate:", position.entrySlisBnbRate);
    console.log("entryBorrowRateBps:", position.entryBorrowRateBps);
    console.log("externalVaultShares:", position.externalVaultShares);
    console.log("status:", uint8(position.status));

    // wrap block
    vm.warp(block.timestamp + 150 seconds);
    vm.roll(block.number + 100);
    vm.prank(USD1_WHALE);
    ERC20(USD1).transfer(address(wallet), 1 * 1e18);
    // unwind
    uint256 collateralReturned =
directFacet.unwindSlisBnbUsd1Market(positionId);
    console.log("Collateral returned from unwind:", collateralReturned);

```

```

// check status again
position = managementFacet.getListPosition(positionId);
assertEq(uint8(position.status), uint8(ListaPositionStatus.CLOSED));
console.log("totalCollateral:", position.totalCollateral);
console.log("totalBorrow:", position.totalBorrowed);
console.log("lastHealthFactor:", position.lastHealthFactor);
console.log("entrySlisBnbRate:", position.entrySlisBnbRate);
console.log("entryBorrowRateBps:", position.entryBorrowRateBps);
console.log("externalVaultShares:", position.externalVaultShares);
console.log("status:", uint8(position.status));

// create another positoin without initialAmount should be successful
positionId = managementFacet.createListaPosition(pos_params);
assertEq(positionId, 1, "positionId should be 1");
}

// deposit to BNB Vault
function test_strategy_BNB_VAULT() public {
    // 1 config strategy, no market intergated
    managementFacet.configureStrategyInfo(
        ListaStrategyType.BNB_VAULT,
        ListaCollateralType.WBNB,
        ListaBorrowType.NONE
    );
    // 2 create a position
    uint256 initialAmount = 1 ether;
    vm.deal(address(this), initialAmount * 2);
    CreateListaPositionParams memory pos_params =
    CreateListaPositionParams({
        strategyType: ListaStrategyType.BNB_VAULT,
        collateralType: ListaCollateralType.WBNB,
        borrowType: ListaBorrowType.NONE,
        initialAmount: initialAmount,
        targetLoops: 1,
        targetLtvBps: 6000,
        slippageBps: DEFAULT_SLIPPAGE,
        pendleMarket: address(0),
        pendleMinPtOut: 0,
        externalVault: address(0),
        useAsterProtocol: false,
        launchpoolDelegate: address(0)
    });

    uint256 positionId = managementFacet.createListaPosition{value:
initialAmount}(pos_params);
    assertEq(positionId, 0);
    // execute position
    directFacet.executeListaBnbVaultDeposit(positionId);
    // check status of position
    SLYWalletListaStorage.ListaPosition memory position =
managementFacet.getListPosition(positionId);
    assertEq(position.id, 0);
    assertEq(position.targetLoops, 1);
    assertEq(position.currentLoops, 1);
    assertEq(uint8(position.status), uint8(ListaPositionStatus.ACTIVE));
    console.log("totalCollateral:", position.totalCollateral);
    console.log("totalBorrow:", position.totalBorrowed);
    console.log("lastHealthFactor:", position.lastHealthFactor);
    console.log("entrySlisBnbRate:", position.entrySlisBnbRate);
    console.log("entryBorrowRateBps:", position.entryBorrowRateBps);
    console.log("externalVaultShares:", position.externalVaultShares);

    // wrap block
    vm.warp(block.timestamp + 150 seconds);
    vm.roll(block.number + 100);

    // unwind partial
    uint256 collateralReturned =
directFacet.unwindBnbVault(positionId, 3000);
    console.log("Collateral returned from unwind:", collateralReturned);
    position = managementFacet.getListPosition(positionId);
    assertEq(position.id, 0);
    assertEq(position.targetLoops, 1);
    assertEq(position.currentLoops, 1);
    console.log("totalCollateral:", position.totalCollateral);
    console.log("totalBorrow:", position.totalBorrowed);

```

```

        console.log("lastHealthFactor:", position.lastHealthFactor);
        console.log("entrySlisBnbRate:", position.entrySlisBnbRate);
        console.log("entryBorrowRateBps:", position.entryBorrowRateBps);
        console.log("externalVaultShares:", position.externalVaultShares);
        console.log("status:", uint8(position.status));
        assertEq(uint8(position.status),
uint8(ListaPositionStatus.UNWINDING));
        // unwind all
        collateralReturned = directFacet.unwindBnbVault(positionId,10000);
        console.log("Collateral returned from unwind:", collateralReturned);
        position = managementFacet.getListPosition(positionId);
        assertEq(position.id, 0);
        assertEq(position.targetLoops, 1);
        assertEq(position.currentLoops, 1);
        console.log("totalCollateral:", position.totalCollateral);
        console.log("totalBorrow:", position.totalBorrowed);
        console.log("lastHealthFactor:", position.lastHealthFactor);
        console.log("entrySlisBnbRate:", position.entrySlisBnbRate);
        console.log("entryBorrowRateBps:", position.entryBorrowRateBps);
        console.log("externalVaultShares:", position.externalVaultShares);
        console.log("status:", uint8(position.status));
        assertEq(uint8(position.status), uint8(ListaPositionStatus.CLOSED));

        // create another positoin without initialAmount should be successful
        positionId = managementFacet.createListaPosition(pos_params);
        assertEq(positionId, 1,"positionId should be 1");
    }

    // BTC/ETH → BNB Staking
    function test_strategy_ETH_TO_BNB() public {
        managementFacet.configureListaSafetyParams(
            1.25e18,
            1.4e18,
            1.15e18,
            0.001 ether,
            5,
            8000
        );
        // 1. config market
        MarketParams memory params = MarketParams({
            loanToken: WBNB,
            collateralToken: ETH,
            oracle: ORACLE_ADAPTOR_V2,
            irm: IRM,
            lltv: 0.8e18 // 80%
        });
        managementFacet.addListaMarketConfig(
            params,MAX_LTV_BPS,MIN_LTV_BPS
        );
        // 2. config strategy
        ListaStrategyType strategyType = ListaStrategyType.BTC_ETH_TO_BNB;
        ListaCollateralType collateralType = ListaCollateralType.ETH;
        ListaBorrowType borrowType = ListaBorrowType.WBNB;
        managementFacet.configureStrategyInfoWithMarket(
            strategyType,
            collateralType,
            borrowType,
            params
        );
        // 3. create a positon
        uint256 initialAmount = 0.01 ether;
        vm.prank(ETH_WHALE);
        ERC20(ETH).transfer(address(wallet), initialAmount);
        CreateListaPositionParams memory pos_params =
CreateListaPositionParams({
            strategyType: ListaStrategyType.BTC_ETH_TO_BNB,
            collateralType: ListaCollateralType.ETH,
            borrowType: ListaBorrowType.WBNB,
            initialAmount: initialAmount,
            targetLoops: 5,
            targetLtvBps:7500,
            slippageBps: DEFAULT_SLIPPAGE,
            pendleMarket:address(0),
            pendleMinPtOut:0,
            externalVault:address(0),

```

```

        useAsterProtocol:false,
        launchpoolDelegate:address(0)
    });
    uint256 positionId = managementFacet.createListaPosition(pos_params);
    assertEq(positionId, 0);
    // show the assets in the vault
    Id id = params.id();
    (uint128 totalSupplyAssets,
    uint128 totalSupplyShares,
    uint128 totalBorrowAssets,
    uint128 totalBorrowShares,
    uint128 lastUpdate,
    uint128 fee) = IMoolah(MOOLAH).market(id);
    console.log("Vault before execution - totalSupplyAssets:",
log_float(totalSupplyAssets, 18));
    console.log("Vault before execution - totalSupplyShares:",
log_float(totalSupplyShares, 18));
    console.log("Vault before execution - totalBorrowAssets:",
log_float(totalBorrowAssets, 18));
    console.log("Vault before execution - totalBorrowShares:",
log_float(totalBorrowShares, 18));
    console.log("Vault before execution - lastUpdate:", lastUpdate);
    console.log("Vault before execution - fee:", log_float(fee, 18));
    // 4. execute position
    directFacet.executeBtcEthToBnb(positionId);
    // 5. check the status of position
    SLYWalletListaStorage.ListaPosition memory position =
managementFacet.getListPosition(positionId);
    assertEq(position.id, 0);
    assertEq(position.targetLoops, 5);
    assertEq(position.currentLoops, 1);
    assertEq(uint8(position.status), uint8(ListaPositionStatus.ACTIVE));
    console.log("totalCollateral:", position.totalCollateral);
    console.log("totalBorrow:", position.totalBorrowed);
    console.log("lastHealthFactor:", position.lastHealthFactor);
    console.log("entrySlisBnbRate:", position.entrySlisBnbRate);
    console.log("entryBorrowRateBps:", position.entryBorrowRateBps);
    console.log("externalVaultShares:", position.externalVaultShares);
    // show the balance of slisBNB
    uint256 balance = ERC20(SLIS_BNB).balanceOf(address(wallet));
    console.log("SLIS_BNB balance:", balance);

    // wrap block
    vm.warp(block.timestamp + 150 seconds);
    vm.roll(block.number + 100);
    // 6. unwind should be successful
    uint256 collateralReturned =
directFacet.unwindBtcEthToBnb(positionId);
    console.log("Collateral returned from unwind:", collateralReturned);

    // 7. create another positoin without initialAmount should be
successful
    positionId = managementFacet.createListaPosition(pos_params);
    assertEq(positionId, 1, "positionId should be 1");
}

function test_strategy_BTC_TO_BNB() public {
    // 1. config market
    MarketParams memory params = MarketParams({
        loanToken: WBNB,
        collateralToken: BTCTB,
        oracle: ORACLE_ADAPTOR_V2,
        irm: IRM,
        lltv: 0.8e18 // 80%
    });
    managementFacet.addListaMarketConfig(
        params, MAX_LTV_BPS, MIN_LTV_BPS
    );

    // 2. config strategy
    ListaStrategyType strategyType = ListaStrategyType.BTC_ETH_TO_BNB;
    ListaCollateralType collateralType = ListaCollateralType.BTCTB;
    ListaBorrowType borrowType = ListaBorrowType.WBNB;
    managementFacet.configureStrategyInfoWithMarket(
        strategyType,

```

```

        collateralType,
        borrowType,
        params
    );

    // 3. create a positon
    uint256 initialAmount = 0.1 ether;
    vm.prank(BTCB_WHALE);
    ERC20(BTCB).transfer(address(wallet), initialAmount);
    CreateListaPositionParams memory pos_params =
    CreateListaPositionParams({
        strategyType: ListaStrategyType.BTC_ETH_TO_BNB,
        collateralType: ListaCollateralType.BTCB,
        borrowType: ListaBorrowType.WBNB,
        initialAmount: initialAmount,
        targetLoops: 5,
        targetLtvBps: 7500,
        slippageBps: DEFAULT_SLIPPAGE,
        pendleMarket: address(0),
        pendleMinPtOut: 0,
        externalVault: address(0),
        useAsterProtocol: false,
        launchpoolDelegate: address(0)
    });
    uint256 positionId = managementFacet.createListaPosition(pos_params);
    assertEq(positionId, 0);
    // show the assets in the vault
    Id id = params.id();
    (uint128 totalSupplyAssets,
    uint128 totalSupplyShares,
    uint128 totalBorrowAssets,
    uint128 totalBorrowShares,
    uint128 lastUpdate,
    uint128 fee) = IMoolah(MOOLAH).market(id);
    console.log("Vault before execution - totalSupplyAssets:",
    log_float(totalSupplyAssets, 18));
    console.log("Vault before execution - totalSupplyShares:",
    log_float(totalSupplyShares, 18));
    console.log("Vault before execution - totalBorrowAssets:",
    log_float(totalBorrowAssets, 18));
    console.log("Vault before execution - totalBorrowShares:",
    log_float(totalBorrowShares, 18));
    console.log("Vault before execution - lastUpdate:", lastUpdate);
    console.log("Vault before execution - fee:", log_float(fee, 18));

    // 4. execute position
    directFacet.executeBtcEthToBnb(positionId);
    // 5. check the status of position
    SLYWalletListaStorage.ListaPosition memory position =
    managementFacet.getListPosition(positionId);
    assertEq(position.id, 0);
    assertEq(position.targetLoops, 5);
    assertEq(position.currentLoops, 1);
    assertEq(uint8(position.status), uint8(ListaPositionStatus.ACTIVE));
    console.log("totalCollateral:", position.totalCollateral);
    console.log("totalBorrow:", position.totalBorrowed);
    console.log("lastHealthFactor:", position.lastHealthFactor);
    console.log("entrySlisBnbRate:", position.entrySlisBnbRate);
    console.log("entryBorrowRateBps:", position.entryBorrowRateBps);
    console.log("externalVaultShares:", position.externalVaultShares);
    // check the balance of slisBNB
    uint256 balance = ERC20(SLIS_BNB).balanceOf(address(wallet));
    console.log("SLIS_BNB balance:", balance);

    // 6. wrap block
    vm.warp(block.timestamp + 150 seconds);
    vm.roll(block.number + 100);
    // 7. unwind should be successful
    uint256 collateralReturned =
    directFacet.unwindBtcEthToBnb(positionId);
    console.log("Collateral returned from unwind:", collateralReturned);

    // 8. create another positoin without initialAmount should be
    successful
    positionId = managementFacet.createListaPosition(pos_params);
    assertEq(positionId, 1, "positionId should be 1");

```

```

}

// Smart Lending
function test_strategy_SMART_LENDING() public {
    // 1. deploy mock Lending contract
    MockERC20 share_token = new MockERC20("SHARE", "SHE",18);
    MockSmartLending mockLending = new MockSmartLending(SLIS_BNB,
address(share_token));
    managementFacet.configureSmartLending(address(mockLending));

    // 2. config strategy
    ListaStrategyType strategyType = ListaStrategyType.SMART_LENDING;
    ListaCollateralType collateralType = ListaCollateralType.SLISBNB;
    ListaBorrowType borrowType = ListaBorrowType.NONE;

    managementFacet.configureStrategyInfo(
        strategyType,collateralType,borrowType
    );

    // 3. create a positon
    uint256 initialAmount = 1 ether;
    vm.deal(address(this), initialAmount * 2);
    CreateListaPositionParams memory pos_params =
CreateListaPositionParams({
    strategyType: ListaStrategyType.SMART_LENDING,
    collateralType: ListaCollateralType.SLISBNB,
    borrowType: ListaBorrowType.NONE,
    initialAmount: initialAmount,
    targetLoops: 1,
    targetLtvBps:6000,
    slippageBps: DEFAULT_SLIPPAGE,
    pendleMarket:address(0),
    pendleMinPtOut:0,
    externalVault:address(0),
    useAsterProtocol:false,
    launchpoolDelegate:address(0)
});
    uint256 positionId = managementFacet.createListaPosition(value:
initialAmount)(pos_params);
    assertEq(positionId, 0);
    // 4. execute position
    directFacet.executeSmartLending(positionId);
    // 5. check status
    SLYWalletListaStorage.ListaPosition memory position =
managementFacet.getListPosition(positionId);
    assertEq(position.id, 0);
    assertEq(position.targetLoops, 1);
    assertEq(position.currentLoops, 1);
    assertEq(uint8(position.status), uint8(ListaPositionStatus.ACTIVE));
    console.log("totalCollateral:", position.totalCollateral);
    console.log("smartLendingLpShares:", position.smartLendingLpShares);
    console.log("entrySlisBnbRate:", position.entrySlisBnbRate);
    // 6. wrap block
    vm.warp(block.timestamp + 150 seconds);
    vm.roll(block.number + 100);
    // 7. unwind
    uint256 collateralReturned =
directFacet.unwindSmartLending(positionId,3000);
    console.log("Collateral returned from unwind:", collateralReturned);

    collateralReturned = directFacet.unwindSmartLending(positionId,10000);
    console.log("Collateral returned from unwind:", collateralReturned);

    uint256 slisBNB_balance = ERC20(SLIS_BNB).balanceOf(address(wallet));
    console.log("SLIS_BNB balance after unwind:", slisBNB_balance);

    // 8. create another positoin without initialAmount should be
successful
    positionId = managementFacet.createListaPosition(pos_params);
    assertEq(positionId, 1,"positionId should be 1");
}

function test_get_eth_market() public view {
    get_market_params(ETH, WBNB);
    get_market_params(BTCB, WBNB);
}

```

```

    }

    function get_market_params(
        address collateralToken,
        address loanToken
    ) internal view {
        IBNBVault bnbVault = IBNBVault(BNB_VAULT);
        uint256 queueLength = bnbVault.supplyQueueLength();
        for (uint256 i = 0; i < queueLength; i++) {
            bytes32 queueEntry = bnbVault.supplyQueue(i);
            Id id = Id.wrap(queueEntry);
            MarketParams memory params = IMoolah(MOOLAH).idToMarketParams(id);
            if(params.collateralToken == collateralToken && params.loanToken
            == loanToken) {
                console.log("Found market in BNB vault queue at index", i);
                console.log("Market ID:");
                console.logBytes32(queueEntry);
                console.log("Loan Token:", params.loanToken);
                console.log("Collateral Token:", params.collateralToken);
                console.log("Oracle:", params.oracle);
                console.log("IRM:", params.irm);
                console.log("LLTV:", params.lltv);
            }
        }
    }

    function get_Lista_direct_selectors() private pure returns (bytes4[]
memory) {
        bytes4[] memory selectors = new bytes4[](10);
        selectors[0] =
        SLYWalletListaDirectFacet.executeSlisBnbUsd1Market.selector;
        selectors[1] =
        SLYWalletListaDirectFacet.executeSlisBnbLisUsdUsd1WithDelegation.selector;
        selectors[2] =
        SLYWalletListaDirectFacet.executeListaBnbVaultDeposit.selector;
        selectors[3] = SLYWalletListaDirectFacet.executeBtcEthToBnb.selector;
        selectors[4] = SLYWalletListaDirectFacet.executeSmartLending.selector;
        selectors[5] =
        SLYWalletListaDirectFacet.unwindSlisBnbUsd1Market.selector;
        selectors[6] =
        SLYWalletListaDirectFacet.unwindSLISBNB_LISUSD_USD1.selector;
        selectors[7] = SLYWalletListaDirectFacet.unwindBnbVault.selector;
        selectors[8] = SLYWalletListaDirectFacet.unwindBtcEthToBnb.selector;
        selectors[9] = SLYWalletListaDirectFacet.unwindSmartLending.selector;
        return selectors;
    }
}

```

2. ListaTestBase.sol

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import {SlyWalletTestBase} from "../SlyWalletTestBase.sol";
import {IDiamondCut} from "src/common/interfaces/IDiamondCut.sol";
import {SLYWalletListaManagementFacet} from
"src/slywallet/facets/lista/SLYWalletListaManagementFacet.sol";

contract ListaTestBase is SlyWalletTestBase {

    address internal constant LISTA_STAKE_MANAGER =
0x1adB950d8bB3dA4bE104211D5AB038628e477fE6;
    address internal constant MOOLAH =
0x8F73b65B4caAf64FBA2aF91cC5D4a2A1318E5D8C;
    address internal constant BNB_VAULT =
0x57134a64B7cD9F9eb72F8255A671F5Bf2fe3E2d0;
    address internal constant USD1_VAULT =
0xfa27f172e0b6ebcEF9c51ABf817E2cb142FbE627;

```

```

    address internal constant PENDLE_ROUTER =
0x8888888888889758F76e7103c6CbF23ABbF58F946;
    address internal constant PANCAKE_ROUTER =
0x13f4EA83D0bd40E75C822255bc855a974568Dd4;
    address internal constant PANCAKE_FACTORY =
0x0BfbCF9fa4f9C56B0F40a671Ad40E0805A091865;
    uint24 internal constant PANCAKE_STABLE_POOL_FEE = 100; // 0.1% fee for
PancakeSwap Stable pools
    // this is a mint contract of usdf => asUsdf
    address internal constant ASTER_MINTER =
0xdB57a53C428a9faFcbFefFB6dd80d0f427543695;
    // Deprecated: no longer used. Kept only for storage/layout compatibility;
any non-zero placeholder is fine.
    address internal constant ASTER_STAKING =
0x0000000000000000000000000000000000000000Face;
    // this is a provider of listdao, not moolah
    address internal constant LISTA_DAO_SLIS_BNB_PROVIDER =
0xfD31e1C5e5571f8E7FE318f80888Cle6da97819b;
    address internal constant CDP_INTERACTION =
0xB68443Ee3e828baD1526b3e0Bdf2Dfc6b1975ec4; // listdao
    address internal constant CLISBNB =
0x4b30fcAA7945fe9fDEFD2895aae539ba102Ed6F6;

    uint256 internal constant MIN_HEALTH_FACTOR = 1.25e18; // 125% health
factor
    uint256 internal constant REBALANCE_THRESHOLD = 1.4e18; // 140% health
factor
    uint256 internal constant EMERGENCY_THRESHOLD = 1.15e18; // 115% health
factor
    uint256 internal constant MAX_LOOPS = 5; // Max loops
    uint256 internal constant MIN_LOOPS = 1; // Min loops
    uint256 internal constant MAX_LTV_BPS = 8000;
    uint256 internal constant MIN_LTV_BPS = 5000;
    uint256 internal constant REBALANCE_COOLDOWN = 100;
    uint256 internal constant MIN_POSITION_SIZE = 0.1 ether;
    uint256 internal constant DEFAULT_SLIPPAGE = 100;
    uint256 internal MIN_POOL_LIQUIDITY = 10 ether;

    SLYWalletListaManagementFacet internal managementFacet;

    string url = vm.envOr("FORKING_URL", string(""));
    uint256 blockNo = vm.envOr("FORKING_BLOCK", uint256(0));

    function setUp() virtual override public {
        super.setUp();

        IDiamondCut.FacetCut[] memory cut = new IDiamondCut.FacetCut[](1);
        cut[0] = IDiamondCut.FacetCut({
            facetAddress: address(new SLYWalletListaManagementFacet()),
            action: IDiamondCut.FacetCutAction.Add,
            functionSelectors: get_Lista_manager_selectors()
        });
        wallet.diamondCut(cut, address(0), new bytes(0));
        managementFacet =
SLYWalletListaManagementFacet(payable(address(wallet)));
        managementFacet.initializeListaStorage();
        managementFacet.configurePancakeSwap(
            PANCAKE_ROUTER,
            PANCAKE_FACTORY,
            PANCAKE_STABLE_POOL_FEE
        );

        managementFacet.configureAster(
            ASTER_MINTER,
            ASTER_STAKING
        );
        managementFacet.configureCDP(
            LISTA_DAO_SLIS_BNB_PROVIDER,
            CDP_INTERACTION
        );
    }

    function get_Lista_manager_selectors() private pure returns (bytes4[]
memory) {
        // All external functions of SLYWalletListaManagementFacet

```

```

        bytes4[] memory selectors = new bytes4[](32);

        selectors[0] =
        SLYWalletListaManagementFacet.initializeListaStorage.selector;
        selectors[1] =
        SLYWalletListaManagementFacet.decreaseLeverage.selector;
        selectors[2] =
        SLYWalletListaManagementFacet.configureListaSafetyParams.selector;
        selectors[3] =
        SLYWalletListaManagementFacet.addListaMarketConfig.selector;
        selectors[4] =
        SLYWalletListaManagementFacet.updateListaMarketConfig.selector;
        selectors[5] =
        SLYWalletListaManagementFacet.configureStrategyInfo.selector;
        selectors[6] =
        SLYWalletListaManagementFacet.configureStrategyInfoWithMarket.selector;
        selectors[7] =
        SLYWalletListaManagementFacet.isStrategyConfigured.selector;
        selectors[8] =
        SLYWalletListaManagementFacet.createListaPosition.selector;
        selectors[9] =
        SLYWalletListaManagementFacet.updateListaPosition.selector;
        selectors[10] =
        SLYWalletListaManagementFacet.cancelListaPosition.selector;
        selectors[11] =
        SLYWalletListaManagementFacet.hasActiveListaPosition.selector;
        selectors[12] =
        SLYWalletListaManagementFacet.increaseLeverage.selector;
        selectors[13] =
        SLYWalletListaManagementFacet.configureListaOracles.selector;
        selectors[14] =
        SLYWalletListaManagementFacet.getOracleConfig.selector;
        selectors[15] = SLYWalletListaManagementFacet.configureAster.selector;
        selectors[16] =
        SLYWalletListaManagementFacet.configureSmartLending.selector;
        selectors[17] = SLYWalletListaManagementFacet.configureCDP.selector;
        selectors[18] =
        SLYWalletListaManagementFacet.configureDexRouter.selector;
        selectors[19] =
        SLYWalletListaManagementFacet.configurePancakeSwap.selector;
        selectors[20] =
        SLYWalletListaManagementFacet.getPancakeSwapConfig.selector;
        selectors[21] =
        SLYWalletListaManagementFacet.pauseListaOperations.selector;
        selectors[22] =
        SLYWalletListaManagementFacet.unpauseListaOperations.selector;
        selectors[23] =
        SLYWalletListaManagementFacet.setListaFeeConfig.selector;
        selectors[24] =
        SLYWalletListaManagementFacet.setListaExecutorFeeConfig.selector;
        selectors[25] =
        SLYWalletListaManagementFacet.getStrategyInfo.selector;
        selectors[26] =
        SLYWalletListaManagementFacet.isListaInitialized.selector;
        selectors[27] = SLYWalletListaManagementFacet.isListaPaused.selector;
        selectors[28] =
        SLYWalletListaManagementFacet.getListPosition.selector;
        selectors[29] =
        SLYWalletListaManagementFacet.getUserListaPositions.selector;
        selectors[30] =
        SLYWalletListaManagementFacet.getListSafetyParams.selector;
        selectors[31] =
        SLYWalletListaManagementFacet.getListMarketConfig.selector;

        return selectors;
    }
}

```

3. ListaManagementFacet.t.sol

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

```

```

import {ListaTestBase} from "./ListaTestBase.sol";
import {MarketParams, Id, MarketParamsLib} from
"src/common/lista/IListaLending.sol";
import {ISLYWalletListaManagementFacet} from
"src/slywallet/facets/lista/ISLYWalletListaManagementFacet.sol";
import {SLYWalletListaStorage} from
"src/slywallet/facets/lista/SLYWalletListaStorage.sol";
import
{ListaMarketConfig, ListaStrategyType, ListaCollateralType, ListaBorrowType} from
"src/slywallet/facets/lista/ListaTypes.sol";
import {CreateListaPositionParams} from
"src/slywallet/facets/lista/ListaTypes.sol";
import {MockListaStakeManagerNew} from
"src/common/mocks/MockListaStakeManagerNew.sol";

contract ListaManagementFacetTest is ListaTestBase {

    using MarketParamsLib for MarketParams;

    address internal constant IRM =
0xFE7dAe87EBb11a7BEB9F534BB23267992d9cDe7c;
    address internal constant ORACLE_ADAPTOR =
0x21650E416dC6C89486B2E654c86cC2c36c597b58; // slisBNB markets
    address internal constant ORACLE_ADAPTOR_V2 =
0xf3afD82A4071f272F403dC176916141f44E6c750; // BTCB, ETH, asUSDF, etc.

    function setUp() virtual override public {
        super.setUp();
        // need to deploy MockListaStakeManagerNew on address of
LISTA_STAKE_MANAGER
        MockListaStakeManagerNew stakeManager = new
MockListaStakeManagerNew();
        vm.eth(LISTA_STAKE_MANAGER, address(stakeManager).code);
    }

    function test_addListaMarketConfig() public {
        // SLISBNB_USD1
        MarketParams memory params = MarketParams({
            loanToken: USD1,
            collateralToken: SLIS_BNB,
            oracle: ORACLE_ADAPTOR,
            irm: IRM,
            lltv: 0.6e18 // 60%
        });
        bytes32 marketId = keccak256(abi.encode(params));
        vm.expectEmit(true, true, true, true);
        emit ISLYWalletListaManagementFacet.ListaMarketConfigAdded(
            marketId,
            params.collateralToken,
            params.loanToken
        );
        // validate the event emitted and the stored config
        managementFacet.addListaMarketConfig(
            params, 6000, MIN_LTV_BPS
        );
        ListaMarketConfig memory config =
managementFacet.getListaMarketConfig(
            marketId
        );
        // check the stored config
        assertEq(config.marketParams.collateralToken, params.collateralToken);
        assertEq(config.marketParams.loanToken, params.loanToken);
        assertEq(config.marketParams.oracle, params.oracle);
        assertEq(config.marketParams.irm, params.irm);
        assertEq(config.marketParams.lltv, params.lltv);
        assertEq(config.maxLtvBps, 6000);
        assertEq(config.minLtvBps, MIN_LTV_BPS);
        assertEq(config.isActive, true);

        // SLISBNB_WBNB
        params = MarketParams({
            loanToken: WBNB,
            collateralToken: SLIS_BNB,

```

```

        oracle: ORACLE_ADAPTOR,
        irm: IRM,
        lltv: 0.965e18 // 96.5%
    });
    managementFacet.addListaMarketConfig(
        params, MAX_LTV_BPS, MIN_LTV_BPS
    );
    // BTCB_WBNB
    params = MarketParams({
        loanToken: WBNB,
        collateralToken: BTCB,
        oracle: ORACLE_ADAPTOR_V2,
        irm: IRM,
        lltv: 0.8e18 // 80%
    });
    managementFacet.addListaMarketConfig(
        params, MAX_LTV_BPS, MIN_LTV_BPS
    );
    // ETH_WBNB
    params = MarketParams({
        loanToken: WBNB,
        collateralToken: ETH,
        oracle: ORACLE_ADAPTOR_V2,
        irm: IRM,
        lltv: 0.8e18 // 80%
    });
    managementFacet.addListaMarketConfig(
        params, MAX_LTV_BPS, MIN_LTV_BPS
    );
    // ASUSDF_USD1
    params = MarketParams({
        loanToken: USD1,
        collateralToken: ASUSDF,
        oracle: ORACLE_ADAPTOR_V2,
        irm: IRM,
        lltv: 0.915e18 // 91.5%
    });
    managementFacet.addListaMarketConfig(
        params, MAX_LTV_BPS, MIN_LTV_BPS
    );
}

function test_configureStrategyMarketParams() public {
    MarketParams memory params = MarketParams({
        loanToken: USD1,
        collateralToken: SLIS_BNB,
        oracle: ORACLE_ADAPTOR,
        irm: IRM,
        lltv: 0.6e18 // 60%
    });
    managementFacet.addListaMarketConfig(
        params, 6000, MIN_LTV_BPS
    );
    ListaStrategyType strategyType =
ListaStrategyType.SLISBNB_USD1_MARKET;
    ListaCollateralType collateralType = ListaCollateralType.SLISBNB;
    ListaBorrowType borrowType = ListaBorrowType.USD1;
    assertEq(managementFacet.isStrategyConfigured(strategyType,
collateralType, borrowType), false);
    vm.expectEmit(true, true, true, true);
    emit ISLYWalletListaManagementFacet.StrategyMarketParamsConfigured(
        strategyType,
        collateralType,
        borrowType,
        address(0), // inputToken is not used in this strategy, so it can
be set to 0
        params.id32()
    );
    managementFacet.configureStrategyInfoWithMarket(
        strategyType,
        collateralType,
        borrowType,
        params
    );
}

```

```

    assertEq(managementFacet.isStrategyConfigured(strategyType,
collateralType, borrowType), true);
    (ISLYWalletListaStorage.StrategyInfo memory info, ListaMarketConfig
memory marketConfig) = managementFacet.getStrategyInfo(strategyType,
collateralType, borrowType);
    assertEq(info.isRegistered, true);
    assertEq(info.marketId, params.id32());
    assertEq(info.inputToken, address(0)); // bnb
    assertEq(uint8(info.strategyType), uint8(strategyType));
    assertEq(uint8(info.collateralType), uint8(collateralType));
    assertEq(uint8(info.borrowType), uint8(borrowType));

    MarketParams memory storedParams = marketConfig.marketParams;

    assertEq(storedParams.loanToken, params.loanToken);
    assertEq(storedParams.collateralToken, params.collateralToken);
    assertEq(storedParams.oracle, params.oracle);
    assertEq(storedParams.irm, params.irm);
    assertEq(storedParams.lltv, params.lltv);
}

function test_createListaPosition() public {
    // config market
    MarketParams memory market_params = MarketParams({
        loanToken: USD1,
        collateralToken: SLIS_BNB,
        oracle: ORACLE_ADAPTOR,
        irm: IRM,
        lltv: 0.6e18 // 60%
    });
    managementFacet.addListaMarketConfig(
        market_params,6000,MIN_LTV_BPS
    );
    // config strategy
    ListaStrategyType strategyType =
ListaStrategyType.SLISBNB_USD1_MARKET;
    ListaCollateralType collateralType = ListaCollateralType.SLISBNB;
    ListaBorrowType borrowType = ListaBorrowType.USD1;
    managementFacet.configureStrategyInfoWithMarket(
        strategyType,
        collateralType,
        borrowType,
        market_params
    );

    uint256 initialAmount = 1 ether;
    (bool hasActive, uint256 activeId) =
managementFacet.hasActiveListaPosition();
    assertEq(hasActive, false);
    assertEq(activeId, 0);
    // create position of SLISBNB_USD1_MARKET strategy without delegate
    CreateListaPositionParams memory params = CreateListaPositionParams({
        strategyType: ListaStrategyType.SLISBNB_USD1_MARKET,
        collateralType: ListaCollateralType.SLISBNB,
        borrowType: ListaBorrowType.USD1,
        initialAmount: initialAmount,
        targetLoops: 3,
        targetLtvBps:6000,
        slippageBps: DEFAULT_SLIPPAGE,
        pendleMarket:address(0),
        pendleMinPtOut:0,
        externalVault:address(0),
        useAsterProtocol:false,
        launchpoolDelegate:address(0)
    });
    vm.expectEmit(true,true,true,true);
    emit ISLYWalletListaManagementFacet.ListaPositionCreated(
        0,
        address(this),
        strategyType,
        initialAmount,
        params.targetLoops
    );
    uint256 positionId =
managementFacet.createListaPosition{value:initialAmount}(params);
    assertEq(positionId, 0);

```

```

        (hasActive, activeId) = managementFacet.hasActiveListaPosition();
        assertEq(hasActive, true);
        assertEq(activeId, 0);

        // Create another position should be failed, because active position
        already exists
        vm.expectRevert("ListaManagement: Active position already exists");
        positionId = managementFacet.createListaPosition(value: initialAmount)
        (params);
    }
}

```

4. ListaSlisBNBLooping.t.sol

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import {ListaTestBase} from "../ListaTestBase.sol";
import {MarketParams, Id} from "src/common/lista/IListaLending.sol";
import
{ListaMarketConfig, ListaStrategyType, ListaCollateralType, ListaBorrowType, Lista
PositionStatus} from "../../src/slywallet/facets/lista/ListaTypes.sol";
import {CreateListaPositionParams} from
"src/slywallet/facets/lista/ListaTypes.sol";
import {SLYWalletListaBnbLoopingFacet} from
"src/slywallet/facets/lista/SLYWalletListaBnbLoopingFacet.sol";
import {SLYWalletListaStorage} from
"src/slywallet/facets/lista/SLYWalletListaStorage.sol";
import {IDiamondCut} from "src/common/interfaces/IDiamondCut.sol";
import {IMoolah, Id, MarketParamsLib} from "src/common/lista/IListaLending.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {console} from "forge-std/console.sol";

interface IBNBVault {
    function supplyQueueLength() external view returns (uint256);
    function supplyQueue(uint256) external view returns (bytes32);
}

contract ListaSlisBNBLoopingTest is ListaTestBase {
    using MarketParamsLib for MarketParams;
    SLYWalletListaBnbLoopingFacet bnbLoopFacet;
    address internal constant IRM =
0xFE7dAe87Ebb11a7BEB9F534BB23267992d9cDe7c;
    address internal constant ORACLE_ADAPTOR =
0x21650E416dC6C89486B2E654c86cC2c36c597b58; // slisBNB markets
    address internal constant ORACLE_ADAPTOR_V2 =
0xf3afD82A4071f272F403dC176916141f44E6c750;
    address internal constant ORACLE_ADAPTOR_PENDLE =
0xf9135dCC8c520331c73e8538BC22e409Af8BA265;
    address internal constant PT_SLISBNB_MARKET =
0x3C1a3D6B69A866444Fe506F7D38a00a1C2D859C5;
    address internal constant PT_TOKEN =
0xe052823b4aefc6e230FAf46231A57d0905E30AE0; // pt clisBNB
    address internal constant PT_TOKEN_HOLDER =
0x3B7e10fFe65c5A59475055D489F71699F7dabfF4;

    function setUp() virtual override public {
        vm.createSelectFork(url, blockNo);
        super.setUp();
        // add looping facet
        bnbLoopFacet = new SLYWalletListaBnbLoopingFacet();
        IDiamondCut.FacetCut[] memory cut = new IDiamondCut.FacetCut[](1);
        cut[0] = IDiamondCut.FacetCut({
            facetAddress: address(bnbLoopFacet),
            action: IDiamondCut.FacetCutAction.Add,
            functionSelectors: get_Lista_bnb_loop_selectors()
        });
    }
}

```

```

        wallet.diamondCut(cut, address(0), new bytes(0));
        bnbLoopFacet =
    SLYWalletListaBnbLoopingFacet(payable(address(wallet)));
    }

    function test_pt_market_in_bnb_vault_queue() public {
        address token = PT_TOKEN;
        get_pt_market_params(token);
    }

    function get_pt_market_params(address pt_token) public view {
        IBNBVault bnbVault = IBNBVault(BNB_VAULT);
        uint256 queueLength = bnbVault.supplyQueueLength();
        console.log("BNB Vault supply queue length:", queueLength);
        for (uint256 i = 0; i < queueLength; i++) {
            bytes32 queueEntry = bnbVault.supplyQueue(i);
            Id id = Id.wrap(queueEntry);
            MarketParams memory params = IMoolah(MOOLAH).idToMarketParams(id);
            if(params.collateralToken == pt_token) {
                console.log("Found PT market in BNB vault queue at index", i);
                console.log("Market ID:");
                console.logBytes32(queueEntry);
                console.log("Loan Token:", params.loanToken);
                console.log("Collateral Token:", params.collateralToken);
                console.log("Oracle:", params.oracle);
                console.log("IRM:", params.irm);
                console.log("LTV:", params.lltv);
            }
        }
    }

    function test_strategy_SLISBNB_WBNB_MARKET_LOOP() public {
        // 1. config market of SLISBNB_WBNB
        MarketParams memory params = MarketParams({
            loanToken: WBNB,
            collateralToken: SLIS_BNB,
            oracle: ORACLE_ADAPTOR,
            irm: IRM,
            lltv: 0.965e18 // 96.5%
        });
        managementFacet.addListaMarketConfig(
            params, MAX_LTV_BPS, MIN_LTV_BPS
        );
        // 2. config strategy
        ListaStrategyType strategyType =
    ListaStrategyType.SLISBNB_WBNB_MARKET_LOOP;
        ListaCollateralType collateralType = ListaCollateralType.SLISBNB;
        ListaBorrowType borrowType = ListaBorrowType.WBNB;
        managementFacet.configureStrategyInfoWithMarket(
            strategyType,
            collateralType,
            borrowType,
            params
        );
        // 3. create a position
        uint256 initialAmount = 2 ether;
        vm.deal(address(this), initialAmount * 4);
        CreateListaPositionParams memory pos_params =
    CreateListaPositionParams({
        strategyType: ListaStrategyType.SLISBNB_WBNB_MARKET_LOOP,
        collateralType: ListaCollateralType.SLISBNB,
        borrowType: ListaBorrowType.WBNB,
        initialAmount: initialAmount,
        targetLoops: 5,
        targetLtvBps: MAX_LTV_BPS,
        slippageBps: DEFAULT_SLIPPAGE,
        pendleMarket: address(0),
        pendleMinPtOut: 0,
        externalVault: address(0),
        useAsterProtocol: false,
        launchpoolDelegate: address(0)
    });
        uint256 positionId = managementFacet.createListaPosition(value:
    initialAmount)(pos_params);
        assertEq(positionId, 0);
    }

```

```

// 4. execute partial loops
bnbLoopFacet.executeListaLoops(positionId,3);

// 5. check the status of positoin
SLYWalletListaStorage.ListaPosition memory position =
managementFacet.getListPosition(positionId);
assertEq(position.id, 0);
assertEq(position.targetLoops, 5);
assertEq(position.currentLoops, 3);
assertEq(uint8(position.status), uint8(ListaPositionStatus.ACTIVE));
// execute the remain loops
bnbLoopFacet.executeListaLoops(positionId,2);
position = managementFacet.getListPosition(positionId);
assertEq(position.id, 0);
assertEq(position.targetLoops, 5);
assertEq(position.currentLoops, 5);
assertEq(uint8(position.status), uint8(ListaPositionStatus.ACTIVE));

console.log("totalCollateral:", position.totalCollateral);
console.log("totalBorrow:", position.totalBorrowed);
console.log("lastHealthFactor:", position.lastHealthFactor);

// execute again should be failed
vm.expectRevert("ListaBnbLoop: No loops to execute");
bnbLoopFacet.executeListaLoops(positionId,2);

// 6. wrap block
vm.warp(block.timestamp + 150 seconds);
vm.roll(block.number + 100);

// 7. Attempting to unwind the loop
console.log("Attempting to unwind the loop...");
uint256 collateralReturned =
bnbLoopFacet.unwindSlisBnbLoop(positionId,100); // 1% max slippage
console.log(log_float(collateralReturned, 18));
position = managementFacet.getListPosition(positionId);
console.log("totalCollateral:", position.totalCollateral);
console.log("totalBorrow:", position.totalBorrowed);
console.log("lastHealthFactor:", position.lastHealthFactor);
console.log("status:", uint8(position.status));

// 8. create another position while unwinding is successful
initialAmount = initialAmount * 2;
pos_params = CreateListaPositionParams({
    strategyType: ListaStrategyType.SLISBNB_WBNB_MARKET_LOOP,
    collateralType: ListaCollateralType.SLISBNB,
    borrowType: ListaBorrowType.WBNB,
    initialAmount: initialAmount,
    targetLoops: 5,
    targetLtvBps:MAX_LTV_BPS,
    slippageBps: DEFAULT_SLIPPAGE,
    pendleMarket:address(0),
    pendleMinPtOut:0,
    externalVault:address(0),
    useAsterProtocol:false,
    launchpoolDelegate:address(0)
});
positionId = managementFacet.createListaPosition(value: initialAmount)
(pos_params);
assertEq(positionId, 1);

// 9. execute
bnbLoopFacet.executeListaLoops(positionId,3);

// 10. check the status of new position
position = managementFacet.getListPosition(positionId);
assertEq(position.id, 1);
assertEq(position.targetLoops, 5);
assertEq(position.currentLoops, 3);
assertEq(uint8(position.status), uint8(ListaPositionStatus.ACTIVE));
// 11. execute the remain loops
bnbLoopFacet.executeListaLoops(positionId,2);
position = managementFacet.getListPosition(positionId);
assertEq(position.id, 1);
assertEq(position.targetLoops, 5);
assertEq(position.currentLoops, 5);

```

```

assertEq(uint8(position.status), uint8(ListaPositionStatus.ACTIVE));

console.log("totalCollateral:", position.totalCollateral);
console.log("totalBorrow:", position.totalBorrowed);
console.log("lastHealthFactor:", position.lastHealthFactor);

// should be failed while no remain loops
vm.expectRevert("ListaBnbLoop: No loops to execute");
bnbLoopFacet.executeListaLoops(positionId, 2);

// 12. wrap block
vm.warp(block.timestamp + 150 seconds);
vm.roll(block.number + 100);

// 13. unwind the new position
console.log("Attempting to unwind the loop...");
collateralReturned = bnbLoopFacet.unwindsSlisBnbLoop(positionId, 100);
// 1% max slippage
console.log(log_float(collateralReturned, 18));
position = managementFacet.getListPosition(positionId);
console.log("totalCollateral:", position.totalCollateral);
console.log("totalBorrow:", position.totalBorrowed);
console.log("lastHealthFactor:", position.lastHealthFactor);
console.log("status:", uint8(position.status));
}

function test_strategy_PT_SLISBNB_LOOP_leverage_up() public {
    // 1. config market PT_SLISBNB_WBNB
    MarketParams memory params = MarketParams({
        loanToken: WBNB,
        collateralToken: PT_TOKEN,
        oracle: ORACLE_ADAPTOR_PENDLE,
        irm: IRM,
        lltv: 0.965e18 // 96.5%
    });
    managementFacet.addListaMarketConfig(
        params, MAX_LTV_BPS, MIN_LTV_BPS
    );
    // 2. config strategy
    ListaStrategyType strategyType = ListaStrategyType.PT_SLISBNB_LOOP;
    ListaCollateralType collateralType = ListaCollateralType.PT_SLISBNB;
    ListaBorrowType borrowType = ListaBorrowType.WBNB;
    managementFacet.configureStrategyInfoWithMarket(
        strategyType,
        collateralType,
        borrowType,
        params
    );
    uint256 initialAmount = 1 ether;
    vm.deal(address(this), initialAmount * 2);
    // 3. create a position
    CreateListaPositionParams memory pos_params =
    CreateListaPositionParams({
        strategyType: ListaStrategyType.PT_SLISBNB_LOOP,
        collateralType: ListaCollateralType.PT_SLISBNB,
        borrowType: ListaBorrowType.WBNB,
        initialAmount: initialAmount,
        targetLoops: 5,
        targetLtvBps: MAX_LTV_BPS,
        slippageBps: DEFAULT_SLIPPAGE,
        pendleMarket: PT_SLISBNB_MARKET,
        pendleMinPtOut: 0,
        externalVault: address(0),
        useAsterProtocol: false,
        launchpoolDelegate: address(0)
    });
    uint256 positionId = managementFacet.createListaPosition(value:
    initialAmount}(pos_params);
    assertEq(positionId, 0);

    Id id = params.id();
    (uint128 totalSupplyAssets,
    uint128 totalSupplyShares,
    uint128 totalBorrowAssets,
    uint128 totalBorrowShares,

```

```

uint128 lastUpdate,
uint128 fee) = IMoolah(MOOLAH).market(id);
console.log("Vault before execution - totalSupplyAssets:",
log_float(totalSupplyAssets, 18));
console.log("Vault before execution - totalSupplyShares:",
log_float(totalSupplyShares, 18));
console.log("Vault before execution - totalBorrowAssets:",
log_float(totalBorrowAssets, 18));
console.log("Vault before execution - totalBorrowShares:",
log_float(totalBorrowShares, 18));
console.log("Vault before execution - lastUpdate:", lastUpdate);
console.log("Vault before execution - fee:", log_float(fee, 18));

// 4. execute the loops
bnbLoopFacet.executePtSlisBnbLoop(positionId,3);
// 5. check the status of position
SLYWalletListaStorage.ListaPosition memory position =
managementFacet.getListPosition(positionId);
assertEq(position.id, 0);
assertEq(position.targetLoops, 5);
assertEq(position.currentLoops, 3);
assertEq(uint8(position.status), uint8(ListaPositionStatus.ACTIVE));
// execute the remain loops
bnbLoopFacet.executePtSlisBnbLoop(positionId,2);
position = managementFacet.getListPosition(positionId);
assertEq(position.id, 0);
assertEq(position.targetLoops, 5);
assertEq(position.currentLoops, 5);
assertEq(uint8(position.status), uint8(ListaPositionStatus.ACTIVE));
console.log("totalCollateral:", position.totalCollateral);
console.log("totalBorrow:", position.totalBorrowed);
console.log("lastHealthFactor:", position.lastHealthFactor);

// execute again should be failed
vm.expectRevert("ListaBnbLoop: No loops to execute");
bnbLoopFacet.executePtSlisBnbLoop(positionId,2);
assertEq(uint8(position.status), uint8(ListaPositionStatus.ACTIVE));

// wrap block
vm.warp(block.timestamp + 150 seconds);
vm.roll(block.number + 100);

uint256 amount = 0.3 ether;
managementFacet.increaseLeverage(positionId, amount);
position = managementFacet.getListPosition(positionId);
console.log("totalCollateral:", position.totalCollateral);
console.log("totalBorrow:", position.totalBorrowed);
console.log("lastHealthFactor:", position.lastHealthFactor);

uint256 balance =
IERC20(params.collateralToken).balanceOf(address(wallet));
console.log("Wallet collateral token balance after up position:",
log_float(balance, 18));

uint256 slisbnb_balance = IERC20(SLIS_BNB).balanceOf(address(wallet));
console.log("Wallet SLIS_BNB balance after up position:",
log_float(slisbnb_balance, 18));

console.log("Attempting to unwind the loop...");
uint256 collateralReturned =
bnbLoopFacet.unwindPtSlisBnbLoop(positionId,100); // 1% max slippage
console.log(log_float(collateralReturned, 18));
position = managementFacet.getListPosition(positionId);
console.log("totalCollateral:", position.totalCollateral);
console.log("totalBorrow:", position.totalBorrowed);
console.log("lastHealthFactor:", position.lastHealthFactor);
console.log("status:", uint8(position.status));
assertEq(uint8(position.status), uint8(ListaPositionStatus.CLOSED));

balance = IERC20(params.collateralToken).balanceOf(address(wallet));
console.log("Wallet collateral token balance after unwind:",
log_float(balance, 18));

slisbnb_balance = IERC20(SLIS_BNB).balanceOf(address(wallet));

```

```

        console.log("Wallet SLIS_BNB balance after unwind:",
log_float(slisbnb_balance, 18));

        // create another position while unwinding is successful
        positionId = managementFacet.createListaPosition(pos_params);
        assertEq(positionId, 1, "positionId should be 1");

    }

    function test_strategy_PT_SLISBNB_LOOP_leverage_down() public {
        // 1. config market SLISBNB_WBNB
        MarketParams memory params = MarketParams({
            loanToken: WBNB,
            collateralToken: PT_TOKEN,
            oracle: ORACLE_ADAPTOR_PENDLE,
            irm: IRM,
            lltv: 0.965e18 // 96.5%
        });
        managementFacet.addListaMarketConfig(
            params, MAX_LTV_BPS, MIN_LTV_BPS
        );
        // 2. config strategy
        ListaStrategyType strategyType = ListaStrategyType.PT_SLISBNB_LOOP;
        ListaCollateralType collateralType = ListaCollateralType.PT_SLISBNB;
        ListaBorrowType borrowType = ListaBorrowType.WBNB;
        managementFacet.configureStrategyInfoWithMarket(
            strategyType,
            collateralType,
            borrowType,
            params
        );

        uint256 initialAmount = 1 ether;
        vm.deal(address(this), initialAmount * 2);
        // 3. create a position
        CreateListaPositionParams memory pos_params =
        CreateListaPositionParams({
            strategyType: ListaStrategyType.PT_SLISBNB_LOOP,
            collateralType: ListaCollateralType.PT_SLISBNB,
            borrowType: ListaBorrowType.WBNB,
            initialAmount: initialAmount,
            targetLoops: 5,
            targetLtvBps: MAX_LTV_BPS,
            slippageBps: DEFAULT_SLIPPAGE,
            pendleMarket: PT_SLISBNB_MARKET,
            pendleMinPtOut: 0,
            externalVault: address(0),
            useAsterProtocol: false,
            launchpoolDelegate: address(0)
        });
        uint256 positionId = managementFacet.createListaPosition(value:
initialAmount}(pos_params);
        assertEq(positionId, 0);
        // 4. execute the loops
        bnbLoopFacet.executePtSlisBnbLoop(positionId, 3);
        // 5. check the status of position
        SLYWalletListaStorage.ListaPosition memory position =
managementFacet.getListaPosition(positionId);
        assertEq(position.id, 0);
        assertEq(position.targetLoops, 5);
        assertEq(position.currentLoops, 3);
        assertEq(uint8(position.status), uint8(ListaPositionStatus.ACTIVE));
        // execute the remain loops
        bnbLoopFacet.executePtSlisBnbLoop(positionId, 2);
        position = managementFacet.getListaPosition(positionId);
        assertEq(position.id, 0);
        assertEq(position.targetLoops, 5);
        assertEq(position.currentLoops, 5);
        assertEq(uint8(position.status), uint8(ListaPositionStatus.ACTIVE));
        console.log("totalCollateral:", position.totalCollateral);
        console.log("totalBorrow:", position.totalBorrowed);
        console.log("lastHealthFactor:", position.lastHealthFactor);
        // execute again should be failed
        vm.expectRevert("ListaBnbLoop: No loops to execute");
        bnbLoopFacet.executePtSlisBnbLoop(positionId, 2);
        assertEq(uint8(position.status), uint8(ListaPositionStatus.ACTIVE));
    }

```

```

// wrap block
vm.warp(block.timestamp + 150 seconds);
vm.roll(block.number + 100);
vm.prank(PT_TOKEN_HOLDER);
IERC20(PT_TOKEN).transfer(address(wallet), 0.3 ether);

uint256 amount = 0.3 ether;
managementFacet.decreaseLeverage(positionId, amount);
position = managementFacet.getListPosition(positionId);
console.log("totalCollateral:", position.totalCollateral);
console.log("totalBorrow:", position.totalBorrowed);
console.log("lastHealthFactor:", position.lastHealthFactor);

uint256 balance =
IERC20(params.collateralToken).balanceOf(address(wallet));
console.log("Wallet collateral token balance after down position:",
log_float(balance, 18));

uint256 slisbnb_balance = IERC20(SLIS_BNB).balanceOf(address(wallet));
console.log("Wallet SLIS_BNB balance after down position:",
log_float(slisbnb_balance, 18));

console.log("Attempting to unwind the loop...");
uint256 collateralReturned =
bnbLoopFacet.unwindPtSlisBnbLoop(positionId, 100); // 1% max slippage
console.log(log_float(collateralReturned, 18));
position = managementFacet.getListPosition(positionId);
console.log("totalCollateral:", position.totalCollateral);
console.log("totalBorrow:", position.totalBorrowed);
console.log("lastHealthFactor:", position.lastHealthFactor);
console.log("status:", uint8(position.status));

balance = IERC20(params.collateralToken).balanceOf(address(wallet));
console.log("Wallet collateral token balance after unwind:",
log_float(balance, 18));

slisbnb_balance = IERC20(SLIS_BNB).balanceOf(address(wallet));
console.log("Wallet SLIS_BNB balance after unwind:",
log_float(slisbnb_balance, 18));

// create another position while unwinding is successful
positionId = managementFacet.createListaPosition(pos_params);
assertEq(positionId, 1, "positionId should be 1");
}

function test_strategy_PT_SLISBNB_LOOP() public {
// 1. config market SLISBNB_WBNB
MarketParams memory params = MarketParams({
loanToken: WBNB,
collateralToken: PT_TOKEN,
oracle: ORACLE_ADAPTOR_PENDLE,
irm: IRM,
lltv: 0.965e18 // 96.5%
});
managementFacet.addListaMarketConfig(
params, MAX_LTV_BPS, MIN_LTV_BPS
);
// 2. config strategy
ListaStrategyType strategyType = ListaStrategyType.PT_SLISBNB_LOOP;
ListaCollateralType collateralType = ListaCollateralType.PT_SLISBNB;
ListaBorrowType borrowType = ListaBorrowType.WBNB;
managementFacet.configureStrategyInfoWithMarket(
strategyType,
collateralType,
borrowType,
params
);

uint256 initialAmount = 1 ether;
vm.deal(address(this), initialAmount * 2);
// 3. create a position
CreateListaPositionParams memory pos_params =
CreateListaPositionParams({
strategyType: ListaStrategyType.PT_SLISBNB_LOOP,

```

```

        collateralType: ListaCollateralType.PT_SLISBNB,
        borrowType: ListaBorrowType.WBNB,
        initialAmount: initialAmount,
        targetLoops: 5,
        targetLtvBps: MAX_LTV_BPS,
        slippageBps: DEFAULT_SLIPPAGE,
        pendleMarket: PT_SLISBNB_MARKET,
        pendleMinPtOut: 0,
        externalVault: address(0),
        useAsterProtocol: false,
        launchpoolDelegate: address(0)
    });
    uint256 positionId = managementFacet.createListaPosition(value:
initialAmount)(pos_params);
    assertEq(positionId, 0);

    // 4. execute the loops
    bnbLoopFacet.executePtSlisBnbLoop(positionId, 3);
    // 5. check the status of position
    SLYWalletListaStorage.ListaPosition memory position =
managementFacet.getListaPosition(positionId);
    assertEq(position.id, 0);
    assertEq(position.targetLoops, 5);
    assertEq(position.currentLoops, 3);
    assertEq(uint8(position.status), uint8(ListaPositionStatus.ACTIVE));
    // execute the remain loops
    bnbLoopFacet.executePtSlisBnbLoop(positionId, 2);
    position = managementFacet.getListaPosition(positionId);
    assertEq(position.id, 0);
    assertEq(position.targetLoops, 5);
    assertEq(position.currentLoops, 5);
    assertEq(uint8(position.status), uint8(ListaPositionStatus.ACTIVE));
    console.log("totalCollateral:", position.totalCollateral);
    console.log("totalBorrow:", position.totalBorrowed);
    console.log("lastHealthFactor:", position.lastHealthFactor);
    // execute again should be failed
    vm.expectRevert("ListaBnbLoop: No loops to execute");
    bnbLoopFacet.executePtSlisBnbLoop(positionId, 2);
    assertEq(uint8(position.status), uint8(ListaPositionStatus.ACTIVE));

    // wrap block
    vm.warp(block.timestamp + 150 seconds);
    vm.roll(block.number + 100);

    // Attempting to unwind the loop
    console.log("Attempting to unwind the loop...");
    uint256 collateralReturned =
bnbLoopFacet.unwindPtSlisBnbLoop(positionId, 100); // 1% max slippage
    console.log(log_float(collateralReturned, 18));
    position = managementFacet.getListaPosition(positionId);
    console.log("totalCollateral:", position.totalCollateral);
    console.log("totalBorrow:", position.totalBorrowed);
    console.log("lastHealthFactor:", position.lastHealthFactor);
    console.log("status:", uint8(position.status));

    uint256 pt_balance =
IERC20(params.collateralToken).balanceOf(address(wallet));
    console.log("Wallet collateral token balance after unwind:",
log_float(pt_balance, 18));

    uint256 slisbnb_balance = IERC20(SLIS_BNB).balanceOf(address(wallet));
    console.log("Wallet SLIS_BNB balance after unwind:",
log_float(slisbnb_balance, 18));

    // create another position while unwinding is successful
    positionId = managementFacet.createListaPosition(pos_params);
    assertEq(positionId, 1, "positionId should be 1");

}

function get_Lista_bnb_loop_selectors() private pure returns (bytes4[]
memory) {
    bytes4[] memory selectors = new bytes4[](7);
    selectors[0] =
SLYWalletListaBnbLoopingFacet.executeListaLoops.selector;

```

```

        selectors[1] =
        SLYWalletListaBnbLoopingFacet.executePtSlisBnbLoop.selector;
        selectors[2] =
        SLYWalletListaBnbLoopingFacet.executeListaLoopsBatch.selector;
        selectors[3] =
        SLYWalletListaBnbLoopingFacet.executeSingleLoopIteration.selector;
        selectors[4] =
        SLYWalletListaBnbLoopingFacet.executeFirstLoopExternal.selector;
        selectors[5] =
        SLYWalletListaBnbLoopingFacet.unwindSlisBnbLoop.selector;
        selectors[6] =
        SLYWalletListaBnbLoopingFacet.unwindPtSlisBnbLoop.selector;

        return selectors;
    }
}

```

5. ListaStableLooping.t.sol

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import {ListaTestBase} from "../ListaTestBase.sol";
import {MarketParams, Id} from "src/common/lista/IListaLending.sol";
import
{ListaMarketConfig, ListaStrategyType, ListaCollateralType, ListaBorrowType, Lista
PositionStatus} from "../../src/slywallet/facets/lista/ListaTypes.sol";
import {CreateListaPositionParams} from
"src/slywallet/facets/lista/ListaTypes.sol";
import {SLYWalletListaStorage} from
"src/slywallet/facets/lista/SLYWalletListaStorage.sol";
import {IDiamondCut} from "src/common/interfaces/IDiamondCut.sol";
import {IMoolah, Id} from "src/common/lista/IListaLending.sol";
import {SLYWalletListaStableLoopingFacet} from
"src/slywallet/facets/lista/SLYWalletListaStableLoopingFacet.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {console} from "forge-std/console.sol";

struct DistributeWithdrawInfo {
    IERC20 withdrawToken;
    uint256 requestWithdrawNo;
    address receipt;
}

interface IWithdrawable {
    function distributeWithdraw(DistributeWithdrawInfo[] calldata
distributeWithdrawInfoList) external;
}

contract ListaStableLoopingTest is ListaTestBase {

    address internal constant IRM =
0xFe7dAe87Ebb11a7BEB9F534BB23267992d9cDe7c;
    address internal constant PT_USDE =
0x67e84bA0196738A59EE58df848A2c16ED2A6A6F3;
    address internal constant PT_USDE_ORACLE =
0x9A7B029b2Fb0fF2CbA4b8d5C0753862fbD027a63;
    bytes32 internal constant PT_USDE_MARKET_ID =
0xdee403936c6f4c4095e68301c0b5ea2d49fa36ed204b61d504f55b280079a36;
    address internal constant PT_USDE_MARKET =
0xB5B56637810E4d090894785993F4CdD6875D927E; // Expired
    address internal constant ORACLE_ADAPTOR_V2 =
0xf3afD82A4071f272F403dC176916141f44E6c750;
    address internal constant ASUSDF_EARN_BOT =
0xB4AACDA6F49b9c140ee1E9b74F5A982364F3D8ae;
    address internal constant ASUSDF_EARN =
0xdB57a53C428a9faFcbFefFB6dd80d0f427543695;
    address internal constant USDFEarn =
0xC271fc70dD9E678ac1AB632f797894fe4BE2C345;

```

```

SLYWalletListaStableLoopingFacet stableLoopFacet;

function setUp() virtual override public {
    vm.createSelectFork(url, blockNo);
    super.setUp();
    // add looping facet
    stableLoopFacet = new SLYWalletListaStableLoopingFacet();
    IDiamondCut.FacetCut[] memory cut = new IDiamondCut.FacetCut[](1);
    cut[0] = IDiamondCut.FacetCut({
        facetAddress: address(stableLoopFacet),
        action: IDiamondCut.FacetCutAction.Add,
        functionSelectors: get_Lista_stable_loop_selectors()
    });
    wallet.diamondCut(cut, address(0), new bytes(0));
    stableLoopFacet =
    SLYWalletListaStableLoopingFacet(payable(address(wallet)));
}

function test_strategy_PT_USDE_LOOP() public {
    // 1. config market params
    MarketParams memory params = MarketParams({
        loanToken: USD1,
        collateralToken: PT_USDE,
        oracle: PT_USDE_ORACLE,
        irm: IRM,
        lltv: 0.965e18 // 96.5%
    });
    managementFacet.addListaMarketConfig(
        params, MAX_LTV_BPS, MIN_LTV_BPS
    );
    // 2. config strategy
    ListaStrategyType strategyType = ListaStrategyType.PT_USDE_LOOP;
    ListaCollateralType collateralType = ListaCollateralType.PT_USDE;
    ListaBorrowType borrowType = ListaBorrowType.USD1;
    managementFacet.configureStrategyInfoWithMarket(
        strategyType,
        collateralType,
        borrowType,
        params
    );
    // 3. create a position
    uint256 initialAmount = 1 ether;
    vm.prank(USDE_WHALE);
    IERC20(USDE).transfer(address(stableLoopFacet), initialAmount);

    CreateListaPositionParams memory pos_params =
    CreateListaPositionParams({
        strategyType: ListaStrategyType.PT_USDE_LOOP,
        collateralType: ListaCollateralType.PT_USDE,
        borrowType: ListaBorrowType.USD1,
        initialAmount: initialAmount,
        targetLoops: 5,
        targetLtvBps: MAX_LTV_BPS,
        slippageBps: DEFAULT_SLIPPAGE,
        pendleMarket: PT_USDE_MARKET,
        pendleMinPtOut: 0,
        externalVault: address(0),
        useAsterProtocol: false,
        launchpoolDelegate: address(0)
    });

    uint256 positionId = managementFacet.createListaPosition(pos_params);
    assertEq(positionId, 0);
    vm.expectRevert();
    // because the market is expired, so the execution should be failed
    stableLoopFacet.executePtUsdeLoop(positionId, 3);
}

function test_strategy_ASTER_USDF_LOOP() public {
    // 1. config market params
    MarketParams memory params = MarketParams({
        loanToken: USD1,
        collateralToken: ASUSDF,

```

```

        oracle: ORACLE_ADAPTOR_V2,
        irm: IRM,
        lltv: 0.915e18 // 91.5%
    });
    managementFacet.addListaMarketConfig(
        params, MAX_LTV_BPS, MIN_LTV_BPS
    );
    // 2. config strategy
    ListaStrategyType strategyType = ListaStrategyType.ASTER_USDF_LOOP;
    ListaCollateralType collateralType = ListaCollateralType.ASUSDF;
    ListaBorrowType borrowType = ListaBorrowType.USD1;
    managementFacet.configureStrategyInfoWithMarket(
        strategyType,
        collateralType,
        borrowType,
        params
    );
    // 3. create a position
    // 3.1 transfer initial tokens to wallet for test
    uint256 initialAmount = 20 ether; // have a limit of min loan amount
    vm.prank(USDT_WHALE);
    IERC20(USDT).transfer(address(stableLoopFacet), initialAmount);

    CreateListaPositionParams memory pos_params =
    CreateListaPositionParams({
        strategyType: ListaStrategyType.ASTER_USDF_LOOP,
        collateralType: ListaCollateralType.ASUSDF,
        borrowType: ListaBorrowType.USD1,
        initialAmount: initialAmount,
        targetLoops: 5,
        targetLtvBps: MAX_LTV_BPS,
        slippageBps: DEFAULT_SLIPPAGE,
        pendleMarket: address(0), // no need for pendle market
        pendleMinPtOut: 0,
        externalVault: address(0),
        useAsterProtocol: true,
        launchpoolDelegate: address(0)
    });

    uint256 positionId = managementFacet.createListaPosition(pos_params);
    assertEq(positionId, 0);
    // 4. execute the position
    stableLoopFacet.executeAsterUsdfLoop(positionId, 3);
    // 5. show the status
    SLYWalletListaStorage.ListaPosition memory position =
    managementFacet.getListaPosition(positionId);
    assertEq(position.id, 0);
    assertEq(position.targetLoops, 5);
    assertEq(position.currentLoops, 3);
    assertEq(uint8(position.status), uint8(ListaPositionStatus.ACTIVE));
    console.log("totalCollateral:", position.totalCollateral);
    console.log("totalBorrow:", position.totalBorrowed);
    console.log("lastHealthFactor:", position.lastHealthFactor);

    // 6. execute the remain loops
    stableLoopFacet.executeAsterUsdfLoop(positionId, 2);
    position = managementFacet.getListaPosition(positionId);
    assertEq(position.id, 0);
    assertEq(position.targetLoops, 5);
    assertEq(position.currentLoops, 5);
    console.log("totalCollateral:", position.totalCollateral);
    console.log("totalBorrow:", position.totalBorrowed);
    console.log("lastHealthFactor:", position.lastHealthFactor);
    assertEq(uint8(position.status), uint8(ListaPositionStatus.ACTIVE));
    // should be revert while no remain loops
    vm.expectRevert("ListaStableLoop: No loops to execute");
    stableLoopFacet.executeAsterUsdfLoop(positionId, 2);

    // wrap timestamp and blocknumber
    vm.warp(block.timestamp + 150 seconds);
    vm.roll(block.number + 100);

    // Attempting to unwind
    console.log("Attempting to unwind the loop...");
    vm.prank(USDT_WHALE);

```

```

        IERC20(USD1).transfer(address(stableLoopFacet), 50 ether); // transfer
some usd1 to wallet for pay debt
        uint256 collateralReturned =
stableLoopFacet.unwindAsterUsdfLoop(positionId);
        console.log("Collateral returned from unwind:", collateralReturned);

        position = managementFacet.getListPosition(positionId);
        console.log("totalCollateral:", position.totalCollateral);
        console.log("totalBorrow:", position.totalBorrowed);
        console.log("status:", uint8(position.status));
        console.log("asterUnstakeRequestId:", position.asterUnstakeRequestId);

        // approve withdraw
        DistributeWithdrawInfo[] memory distributeWithdrawInfoList = new
DistributeWithdrawInfo[](1);
        distributeWithdrawInfoList[0] = DistributeWithdrawInfo({
            withdrawToken: IERC20(ASUSDF),
            requestWithdrawNo: position.asterUnstakeRequestId,
            receipt: address(stableLoopFacet)
        });
        vm.prank(ASUSDF_EARN_BOT);

        IWithdrawable(ASUSDF_EARN).distributeWithdraw(distributeWithdrawInfoList);
        console.log("mock setup done, starting claim...");
        // claim usdf
        uint256 usdf_received =
stableLoopFacet.claimAsterWithdrawal(positionId);
        console.log("usdf received from claim:", usdf_received);
        // claim usdt
        position = managementFacet.getListPosition(positionId);
        console.log("status:", uint8(position.status));
        console.log("asterUnstakeRequestId:", position.asterUnstakeRequestId);

        // mock set the staufs of USDFearn to withdrawable,so it can be
claimable
        distributeWithdrawInfoList = new DistributeWithdrawInfo[](1);
        distributeWithdrawInfoList[0] = DistributeWithdrawInfo({
            withdrawToken: IERC20(USDF),
            requestWithdrawNo: position.asterUnstakeRequestId,
            receipt: address(stableLoopFacet)
        });
        vm.prank(ASUSDF_EARN_BOT);

        IWithdrawable(USDFearn).distributeWithdraw(distributeWithdrawInfoList);

        // withdraw again
        uint256 usdt_balance_before =
IERC20(USDT).balanceOf(address(stableLoopFacet));
        uint256 usdt_received =
stableLoopFacet.claimAsterWithdrawal(positionId);
        uint256 usdt_balance_after =
IERC20(USDT).balanceOf(address(stableLoopFacet));
        console.log("usdt received from claim:", usdt_received);
        assertEq(usdt_received, usdt_balance_after - usdt_balance_before);

        position = managementFacet.getListPosition(positionId);
        assertEq(uint8(position.status), uint8(ListaPositionStatus.CLOSED));
        console.log("status:", uint8(position.status));
        console.log("asterUnstakeRequestId:", position.asterUnstakeRequestId);
        console.log("totalCollateral:", position.totalCollateral);
        console.log("totalBorrow:", position.totalBorrowed);
        console.log("usdfAmount:", position.usdfAmount);
        console.log("asUsdfAmount:", position.asUsdfAmount);

        //create another positoin without initialAmount should be successful
        positionId = managementFacet.createListaPosition(pos_params);
        assertEq(positionId, 1,"positionId should be 1");
    }

    function get_Lista_stable_loop_selectors() internal pure returns (bytes4[]
memory) {
        bytes4[] memory selectors = new bytes4[](5);
        selectors[0] =
SLYWalletListaStableLoopingFacet.executePtUsdeLoop.selector;
        selectors[1] =
SLYWalletListaStableLoopingFacet.executeAsterUsdfLoop.selector;

```

```
        selectors[2] =
        SLYWalletListaStableLoopingFacet.unwindPtUSDELoop.selector;
        selectors[3] =
        SLYWalletListaStableLoopingFacet.unwindAsterUsdfLoop.selector;
        selectors[4] =
        SLYWalletListaStableLoopingFacet.claimAsterWithdrawal.selector;
        return selectors;
    }
}
```

11.2 External Functions Check Points

1. SLYWalletListaManagementFacet.sol.md

File:

src/slywallet/facets/lista/SLYWalletListaManagementFacet.sol

contract SLYWalletListaManagementFacet is ISLYWalletListaManagementFacet, ListaFacetBase

(Custom interfaces exclude members inherited from third-party dependencies)

Index	Custom Interface	StateMutability	Modifier	UnitTest	Miscellaneous
1	initializeListaStorage()		onlyOwner	Passed	
2	configureListaSafetyParams(uint256, uint256, uint256, uint256, uint256, uint256)		onlyOwnerOrAdmin whenInitialized	Passed	
3	addListaMarketConfig(MarketParams, uint256, uint256)		onlyOwnerOrAdmin whenInitialized	Passed	
4	updateListaMarketConfig(MarketParams, uint256, uint256, bool)		onlyOwnerOrAdmin whenInitialized	Passed	
5	configureStrategyInfo(ListaStrategyType, ListaCollateralType, ListaBorrowType)		onlyOwnerOrAdmin whenInitialized	Passed	
6	increaseLeverage(uint256, uint256)		onlyOwnerOrAdmin whenNotPaused validPosition(positionId)	Passed	
7	decreaseLeverage(uint256, uint256)		onlyOwnerOrAdmin whenNotPaused validPosition(positionId)	Passed	
8	configureStrategyInfoWithMarket(ListaStrategyType, ListaCollateralType, ListaBorrowType, MarketParams)		onlyOwnerOrAdmin whenInitialized	Passed	
9	isStrategyConfigured(ListaStrategyType, ListaCollateralType, ListaBorrowType)	view			
10	createListaPosition(CreateListaPositionParams)	payable	onlyOwnerOrAdmin whenInitialized whenNotPaused noActivePosition	Passed	
11	updateListaPosition(uint256, uint256, uint256, uint256)		positionOwner(positionId)	Passed	
12	cancelListaPosition(uint256)		positionOwner(positionId)	Passed	
13	configureListaOracles(address, address, address, uint256)		onlyOwnerOrAdmin whenInitialized	Passed	
14	getOracleConfig()	view			
15	configureAster(address, address)		onlyOwnerOrAdmin whenInitialized	Passed	
16	configureSmartLending(address)		onlyOwnerOrAdmin whenInitialized	Passed	
17	configureCDP(address, address)		onlyOwnerOrAdmin whenInitialized	Passed	
18	configureDexRouter(address, address, address)		onlyOwnerOrAdmin whenInitialized	Passed	
19	configurePancakeSwap(address, address, uint24)		onlyOwnerOrAdmin whenInitialized	Passed	
20	getPancakeSwapConfig()	view			
21	pauseListaOperations()		onlyOwner	Passed	
22	unpauseListaOperations()		onlyOwner	Passed	
23	setListaFeeConfig(address, uint256)		onlyOwner whenInitialized	Passed	
24	setListaExecutorFeeConfig(uint256, uint256, address)		onlyOwner whenInitialized	Passed	
25	isListaInitialized()	view			
26	isListaPaused()	view			
27	getListaPosition(uint256)	view	returns (SLYWalletListaStorage.ListaPosition memory position)		
28	getUserListaPositions(address)	view			
29	getListaSafetyParams()	view	returns (SLYWalletListaStorage.SafetyParams memory params)		
30	getListaMarketConfig(bytes32)	view			
31	hasActiveListaPosition()	view			
32	getStrategyInfo(ListaStrategyType, ListaCollateralType, ListaBorrowType)	view	returns (SLYWalletListaStorage.StrategyInfo memory info, ListaMarketConfig memory marketConfig)		

2. SLYWalletListaStableLoopingFacet.sol.md

File:

src/slywallet/facets/lista/SLYWalletListaStableLoopingFacet.sol

contract: SLYWalletListaStableLoopingFacet is ListaFacetBase

(Custom interfaces exclude members inherited from third-party dependencies)

Index	Custom Interface	StateMutability	Modifier	UnitTest	Miscellaneous
1	executePtUsdeLoop(uint256, uint256)		nonReentrant whenNotPaused validPositionStrategy(positionId, ListaStrategyType.PT_USDE_LOOP)	Passed	
2	executeAsterUsdfLoop(uint256, uint256)		nonReentrant whenNotPaused validPositionStrategy(positionId, ListaStrategyType.ASTER_USDF_LOOP)	Passed	
3	unwindPtUSDELoop(uint256, uint256)		whenNotPaused nonReentrant validPositionStrategy(positionId, ListaStrategyType.PT_USDE_LOOP) onlyOwnerOrAdmin	Passed	
4	unwindAsterUsdfLoop(uint256)		whenNotPaused nonReentrant validPositionStrategy(positionId, ListaStrategyType.ASTER_USDF_LOOP) onlyOwnerOrAdmin	Passed	
5	claimAsterWithdrawal(uint256)		whenNotPaused nonReentrant validPositionStrategy(positionId, ListaStrategyType.ASTER_USDF_LOOP) onlyOwnerOrAdmin	Passed	

3. SLYWalletListaDirectFacet.sol.md

File:

src/slywallet/facets/lista/SLYWalletListaDirectFacet.sol

contract: SLYWalletListaDirectFacet is ListaFacetBase

(Custom interfaces exclude members inherited from third-party dependencies)

Index	Custom Interface	StateMutability	Modifier	UnitTest	Miscellaneous
1	executeSlisBnbUsd1Market(uint256, address)		nonReentrant whenNotPaused validPositionStrategy(positionId, ListaStrategyType.SLISBNB_USD1_MARKET)	Passed	
2	executeSlisBnbLisUsd1WithDelegation(uint256, address, address)		nonReentrant whenNotPaused validPositionStrategy(positionId, ListaStrategyType.SLISBNB_LISUSD_USD1)	Passed	
3	executeListaBnbVaultDeposit(uint256)		nonReentrant whenNotPaused validPositionStrategy(positionId, ListaStrategyType.BNB_VAULT)	Passed	
4	executeBtcEthToBnb(uint256)		nonReentrant whenNotPaused validPositionStrategy(positionId, ListaStrategyType.BTC_ETH_TO_BNB)	Passed	
5	executeSmartLending(uint256)		nonReentrant whenNotPaused validPositionStrategy(positionId, ListaStrategyType.SMART_LENDING)	Passed	
6	unwindSlisBnbUsd1Market(uint256)		whenNotPaused nonReentrant validPositionStrategy(positionId, ListaStrategyType.SLISBNB_USD1_MARKET) onlyOwnerOrAdmin	Passed	
7	unwindSLISBNB_LISUSD_USD1(uint256)		whenNotPaused nonReentrant validPositionStrategy(positionId, ListaStrategyType.SLISBNB_LISUSD_USD1) onlyOwnerOrAdmin	Passed	
8	unwindBnbVault(uint256, uint256)		whenNotPaused nonReentrant validPositionStrategy(positionId, ListaStrategyType.BNB_VAULT) onlyOwnerOrAdmin	Passed	
9	unwindBtcEthToBnb(uint256)		whenNotPaused nonReentrant validPositionStrategy(positionId, ListaStrategyType.BTC_ETH_TO_BNB) onlyOwnerOrAdmin	Passed	
10	unwindSmartLending(uint256, uint256)		whenNotPaused nonReentrant validPositionStrategy(positionId, ListaStrategyType.SMART_LENDING) onlyOwnerOrAdmin	Passed	

4. SLYWalletListaBnbLoopingFacet.sol.md

File:

src/slywallet/facets/lista/SLYWalletListaBnbLoopingFacet.sol

contract SLYWalletListaBnbLoopingFacet is ListaFacetBase

(Custom interfaces exclude members inherited from third-party dependencies)

Index	Custom Interface	StateMutability	Modifier	UnitTest	Miscellaneous
1	executeListaLoops(uint256, uint256)		nonReentrant whenNotPaused validPositionStrategy(positionId, ListaStrategyType.SLISBNB_WBNB_MARKET_LOOP)	Passed	
2	executePtSlisBnbLoop(uint256, uint256)		nonReentrant whenNotPaused validPositionStrategy(positionId, ListaStrategyType.PT_SLISBNB_LOOP)	Passed	
3	executeListaLoopsBatch(uint256[], uint256)		nonReentrant whenInitialized whenNotPaused	Passed	
4	executeSingleLoopIteration(uint256)				Unused
5	executeFirstLoopExternal(uint256)				Unused
6	unwindSlisBnbLoop(uint256, uint256)		whenNotPaused nonReentrant validPositionStrategy(positionId, ListaStrategyType.SLISBNB_WBNB_MARKET_LOOP) onlyOwnerOrAdmin	Passed	
7	unwindPtSlisBnbLoop(uint256, uint256)		whenNotPaused nonReentrant validPositionStrategy(positionId, ListaStrategyType.PT_SLISBNB_LOOP) onlyOwnerOrAdmin	Passed	



<https://medium.com/@FairyproofT>



<https://twitter.com/FairyproofT>



<https://www.linkedin.com/company/fairyproof-tech>



https://t.me/Fairyproof_tech



Reddit: <https://www.reddit.com/user/FairyproofTech>

